

Synchronous FIFO

Digital VLSI Design

Actor: Hoang Bao Long

1. Specifications

Digital VLSI Design

Actor: Hoang Bao Long

1. Specifications

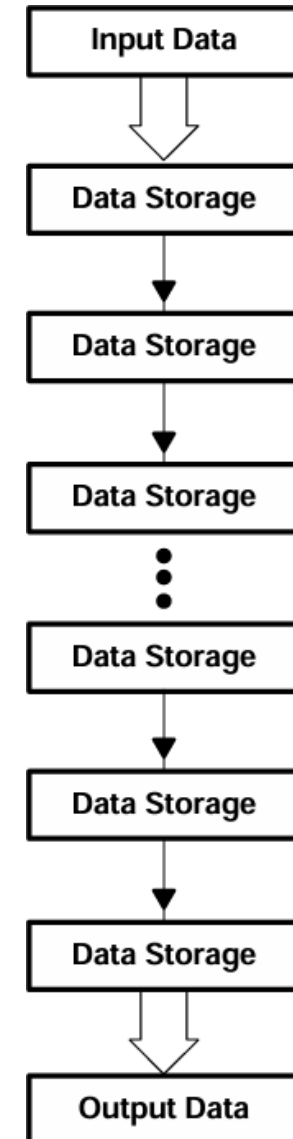
A FIFO is a special type of buffer. The name FIFO stands for first in first out and means that the data written into the buffer first comes out of it first.

Every memory in which the data word that is written in first also comes out first when the memory is read is a first-in first-out memory. There are three kinds of FIFO:

Shift register – FIFO with an invariable number of stored data words and, thus, the necessary synchronism between the read and the write operations because a data word must be read every time one is written.

Exclusive read/write FIFO – FIFO with a variable number of stored data words and, because of the internal structure, the necessary synchronism between the read and the write operations.

Concurrent read/write FIFO – FIFO with a variable number of stored data words and possible asynchronism between the read and the write operation.



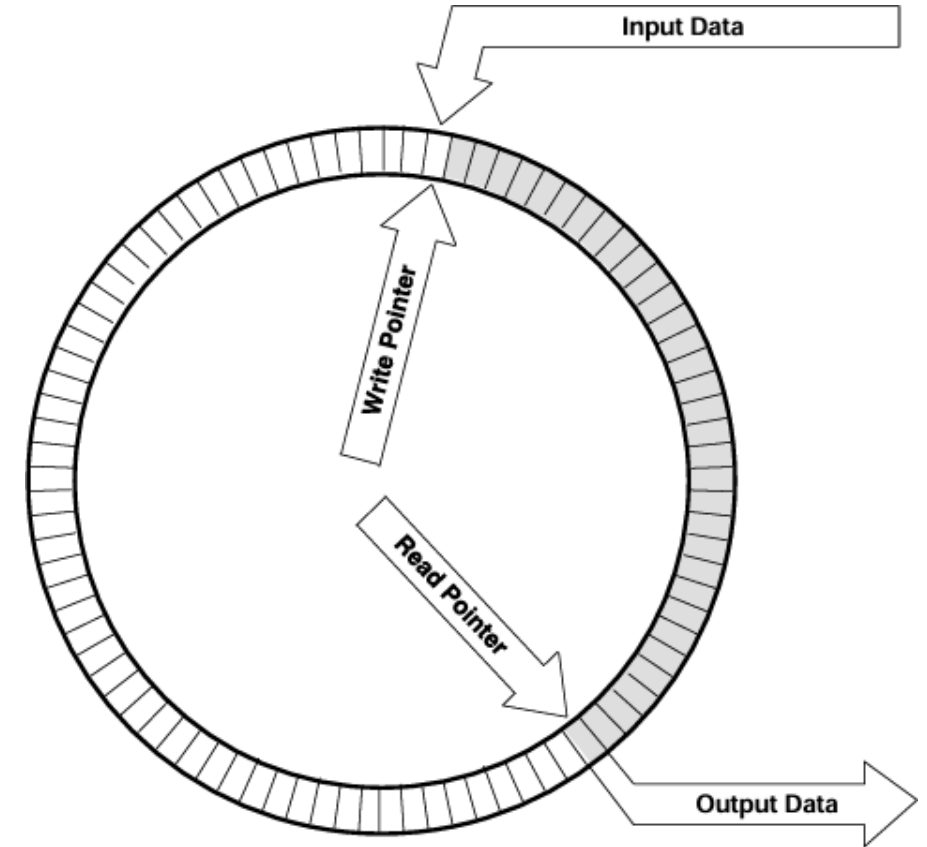
1. Specifications

In concurrent read/write FIFOs, there is no dependence between the writing and reading of data. Simultaneous writing and reading are possible in overlapping fashion or successively. This means that two systems with different frequencies can be connected to the FIFO. The designer need not worry about synchronizing the two systems because this is taken care of in the FIFO. Concurrent read/write FIFOs, depending on the control signals for writing and reading, fall into two groups:

- Synchronous FIFOs.
- Asynchronous FIFOs.

1. Specifications

In a circular FIFO concept, the memory address of the incoming data is in the write pointer. The address of the first data word in the FIFO that is to be read out is in the read pointer. After reset, both pointers indicate the same memory location. After each write operation, the write pointer is set to the next memory location. The reading of a data word sets the read pointer to the next data word that is to be read out. The read pointer constantly follows the write pointer. When the read pointer reaches the write pointer, the FIFO is empty. If the write pointer catches up with the read pointer, the FIFO is full.



2. Design Specifications

2.1. FIFO Interface

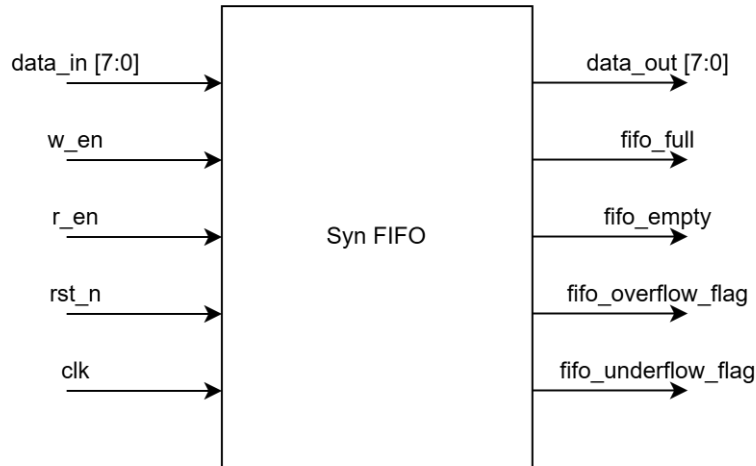
2.2. High Level Block Design

2.3. Low Level Block Design

2.4 RTL Design

2. Design Specifications

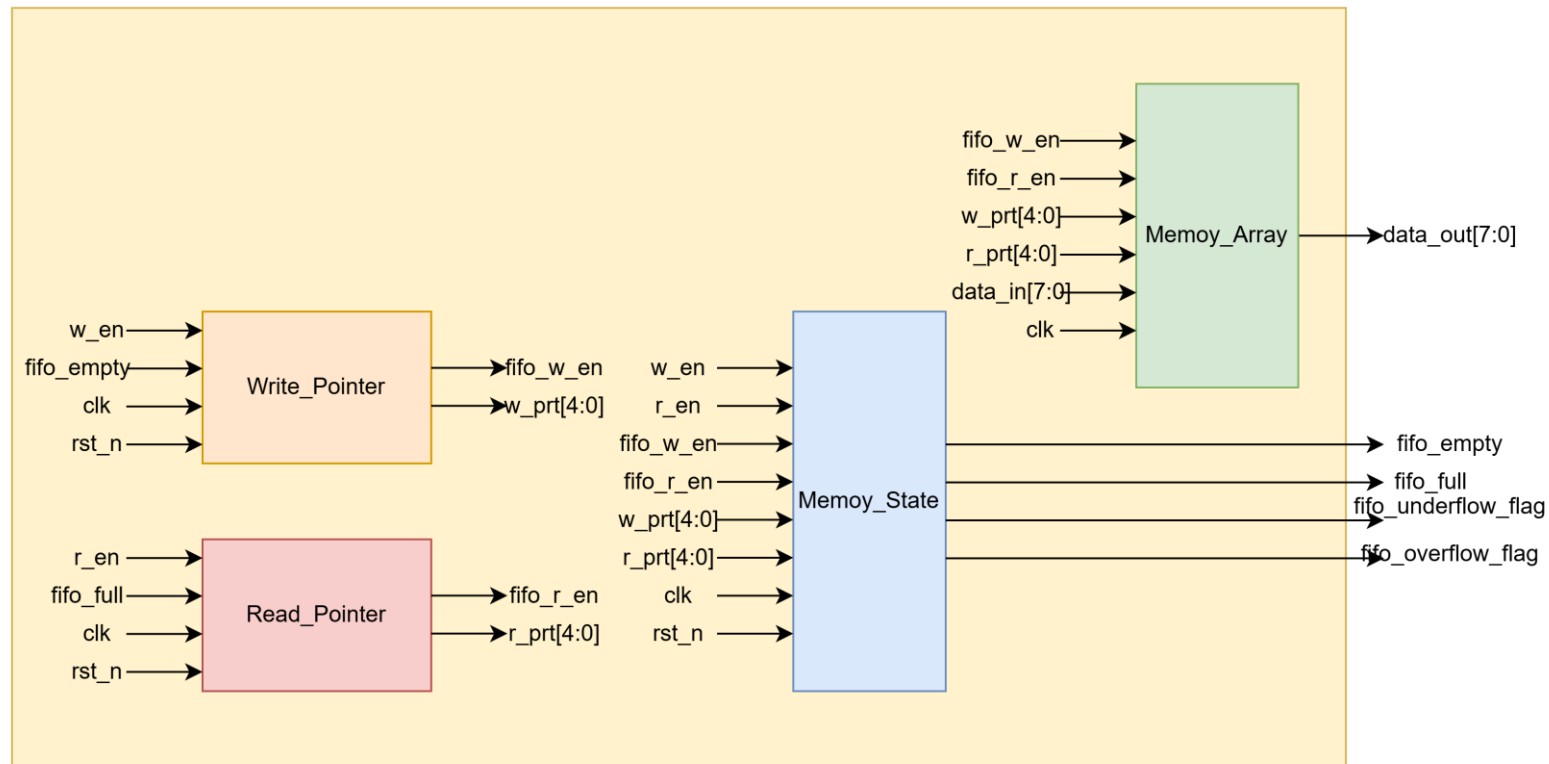
2.1 FIFO Interface



Port	Size	Direction	Description
data_in	8	input	FIFO data input which from RAM,..
clk	1	input	Synchronous clock signal.
rst_n	1	input	Synchronous reset signal active low.
w_en	1	input	Write enable signal.
r_en	1	input	Read enable signal.
data_out	8	output	FIFO data output.
fifo_full	1	output	FIFO full output signal.
fifo_empty	1	output	FIFO empty output signal.
fifo_overflow_flag	1	output	FIFO overflow output signal.
fifo_underflow_flag	1	output	FIFO underflow output signal.

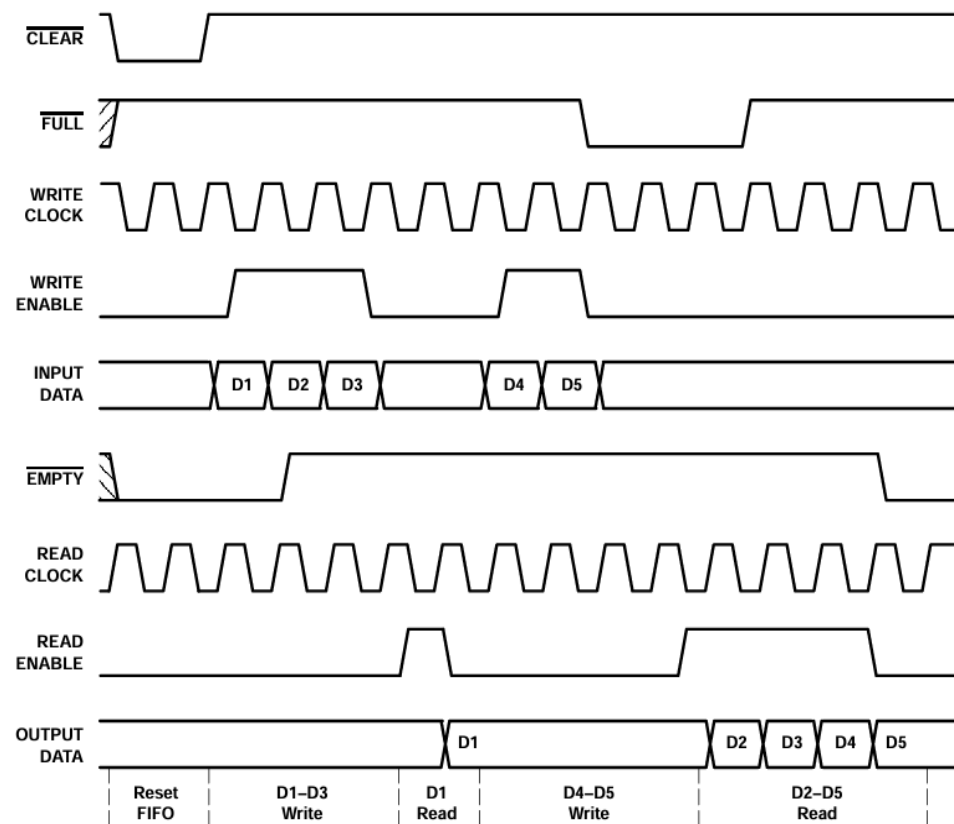
2. Design Specifications

2.1. High Level Block Design



2. Design Specifications

2.1. High Level Block Design

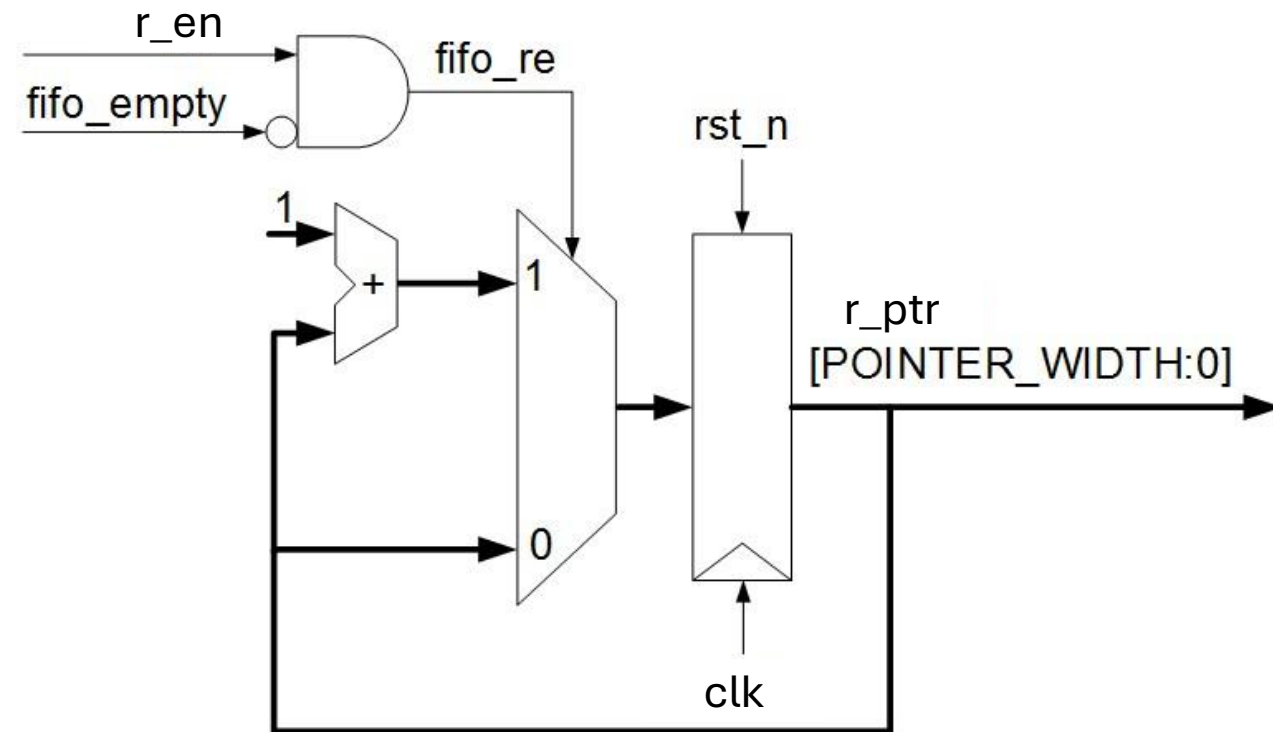
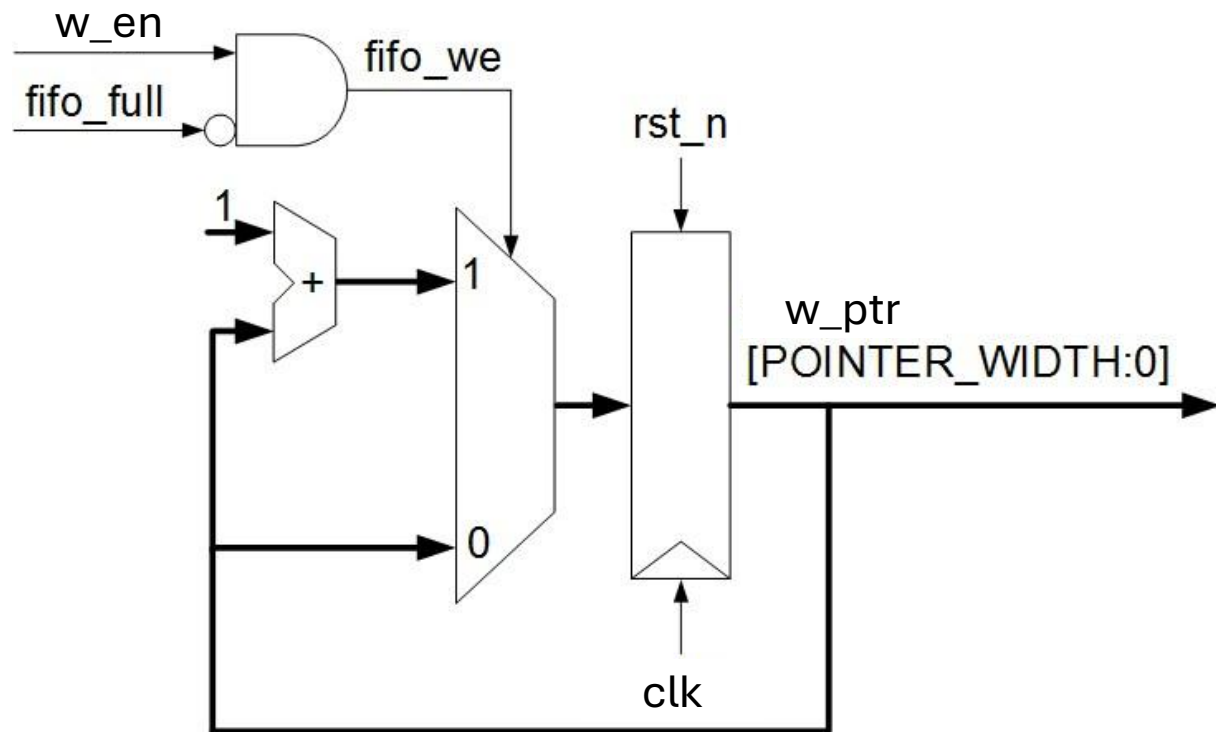


Timing diagram below shows the typical waveform in a synchronous FIFO. WRITE CLOCK and READ CLOCK are free running. The writing of new data into the FIFO is initialized by a low level on the WRITE ENABLE line. The data are written into the FIFO with the next rising edge of WRITE CLOCK. In analogous fashion, the READ ENABLE line controls the reading out of data synchronous with READ CLOCK.

All status lines within the FIFO can be synchronized by the two free-running-clock signals. The FULL line only changes its level synchronously with WRITE CLOCK, even if the change is produced by the reading of a data word. Likewise, the EMPTY signal is synchronized with READ CLOCK. A synchronous FIFO is the only concurrent read/write FIFO in which the status signals are synchronized with the driving logic

2. Design Specifications

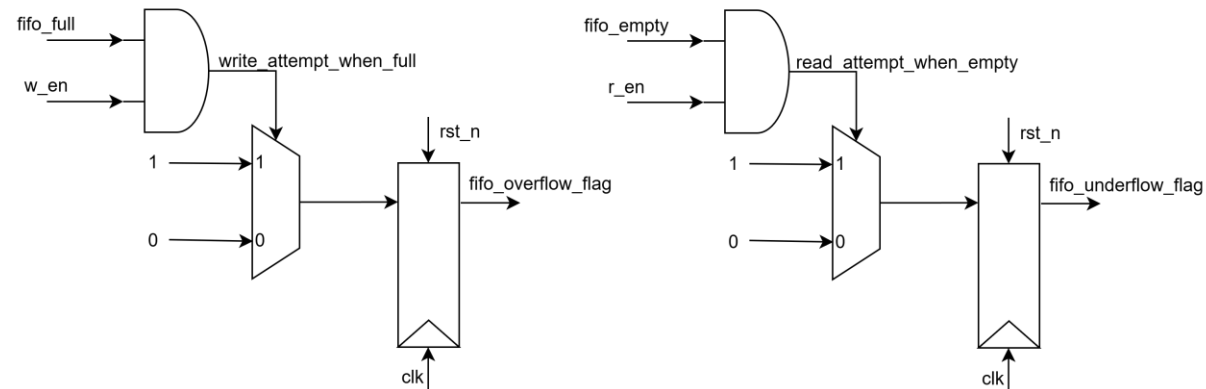
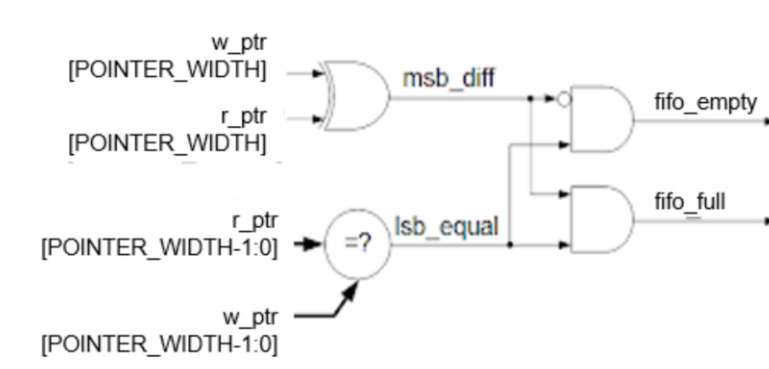
2.3. Low Level Block Design



FIFO Read/Write pointer circuits

2. Design Specifications

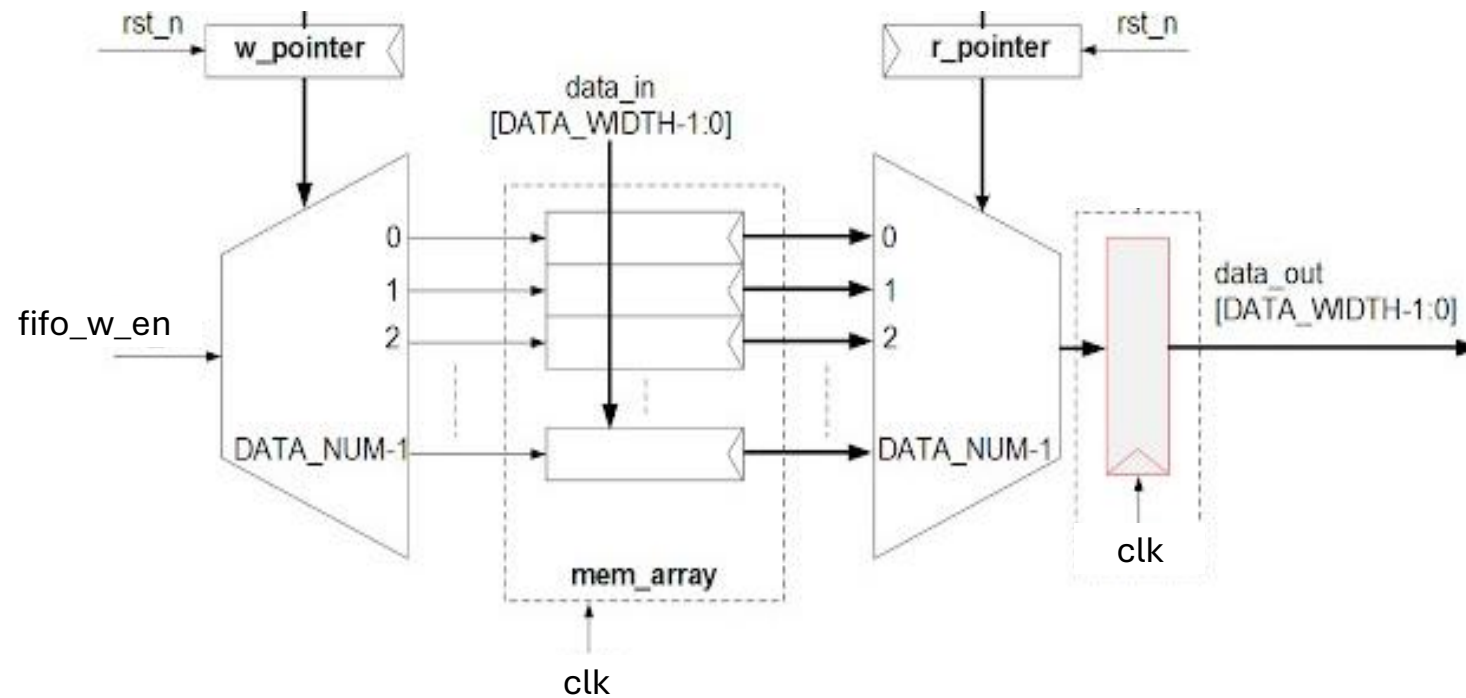
2.3. Low Level Block Design



Memory State

2. Design Specifications

2.3. Low Level Block Design



Memory Array Block

2. Design Specifications

2.4. RTL Design

```
module Synchronous_FIFO_memory #(
    parameter DATASIZE = 8,
    parameter DEPTH = 16,
    parameter PTR_WIDTH = $clog2(DEPTH)
)(
    input wire          clk,
    input wire          rst_n,
    input wire          w_en,
    input wire          r_en,
    input wire [DATASIZE-1:0] data_in,
    output wire [DATASIZE-1:0] data_out,
    output wire         fifo_full,
    output wire         fifo_empty,
    output wire         fifo_overflow_flag,
    output wire         fifo_underflow_flag
);
    wire [PTR_WIDTH:0] w_ptr, r_ptr;
    wire fifo_w_en, fifo_r_en;

    Write_pointer #(.DATASIZE(DATASIZE), .DEPTH(DEPTH)) u_write_ptr (
        .w_en(w_en),
        .fifo_full(fifo_full),
        .clk(clk),
        .rst_n(rst_n),
        .fifo_w_en(fifo_w_en),
        .w_ptr(w_ptr)
    );
```

```
    Read_pointer #(.DATASIZE(DATASIZE), .DEPTH(DEPTH)) u_read_ptr (
        .r_en(r_en),
        .fifo_empty(fifo_empty),
        .clk(clk),
        .rst_n(rst_n),
        .fifo_r_en(fifo_r_en),
        .r_ptr(r_ptr)
    );
```

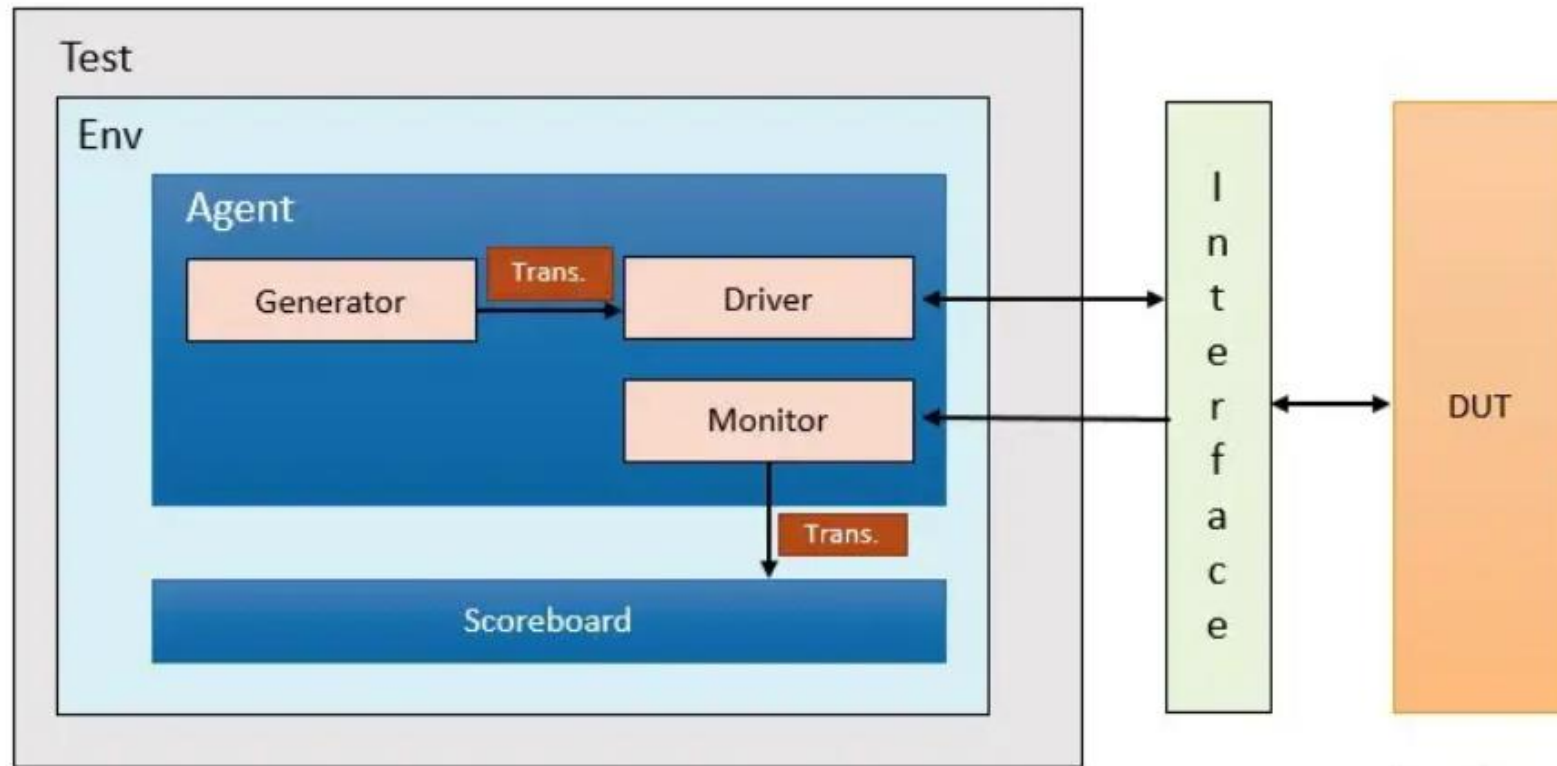
```
    Memory_Array #(.DATASIZE(DATASIZE), .DEPTH(DEPTH)) u_mem (
        .clk(clk),
        .fifo_w_en(fifo_w_en),
        .w_ptr(w_ptr),
        .r_ptr(r_ptr),
        .data_in(data_in),
        .fifo_r_en(fifo_r_en),
        .data_out(data_out)
    );
```

```
    Memory_State #(.DATASIZE(DATASIZE), .DEPTH(DEPTH)) u_state (
        .clk(clk),
        .rst_n(rst_n),
        .w_en(w_en),
        .r_en(r_en),
        .fifo_w_en(fifo_w_en),
        .fifo_r_en(fifo_r_en),
        .w_ptr(w_ptr),
        .r_ptr(r_ptr),
        .fifo_full(fifo_full),
        .fifo_empty(fifo_empty),
        .fifo_overflow_flag(fifo_overflow_flag),
        .fifo_underflow_flag(fifo_underflow_flag)
    );
```

endmodule

3. Design Verification

3. Design Verification



Testbench Environment

3. Design Verification

```
interface fifo_if #(parameter DATASIZE = 8 ) (input bit clk, input bit rst_n);  
    logic w_en;  
    logic r_en;  
    logic [DATASIZE-1:0] data_in;  
    logic [DATASIZE-1:0] data_out;  
    logic fifo_full;  
    logic fifo_empty;  
    logic fifo_overflow_flag;  
    logic fifo_underflow_flag;  
endinterface
```

FIFO Interface

3. Design Verification

```
class transaction;

    // Kiểu giao dịch: "WRITE" hoặc "READ"
    string tr_type;

    // Dữ liệu và tín hiệu FIFO
    randc bit [7:0] data_in;
    bit [7:0] data_out;
    bit fifo_full;
    bit fifo_empty;
    bit fifo_overflow_flag;
    bit fifo_underflow_flag;
    // Cờ điều khiển
    rand bit w_en;
    rand bit r_en;

    constraint c1 { !(w_en && r_en); }
endclass
```

transaction

```
class generator;
    int count;
    mailbox gen_to_drv = new();
    transaction tr;

    function new(mailbox gen_to_drv);
        this.gen_to_drv = gen_to_drv;
    endfunction

    task run;
        repeat(count) begin
            tr = new();
            void'(tr.randomize() with {
                w_en dist {1:=60, 0:=40};
                r_en dist {1:=40, 0:=60};
                data_in inside {[0:255]};
            });
            gen_to_drv.put(tr);
        end
    endtask
endclass
```

generator

3. Design Verification

```
class driver;
    virtual fifo_if vif;
    mailbox gen_to_drv = new();
    transaction tr;

    function new(mailbox gen_to_drv, virtual fifo_if vif);
        this.gen_to_drv = gen_to_drv;
        this.vif = vif;
    endfunction

    task run;
        wait(vif.rst_n == 1); // Đảm bảo DUT thoát reset
        forever begin
            gen_to_drv.get(tr); // Nhận transaction từ generator

            // Chờ cạnh clock để gán dữ liệu vào DUT
            @(posedge vif.clk);

            vif.w_en = tr.w_en;
            vif.r_en = tr.r_en;

            if (!vif.fifo_full && tr.w_en) begin
                vif.data_in = tr.data_in; // Gán dữ liệu ngay
            end

            if (!vif.fifo_empty && tr.r_en) begin
                @(posedge vif.clk); // chờ dữ liệu valid
                vif.r_en <= 1;
                tr.data_out = vif.data_out;
                vif.r_en <= 0;
                gen_to_drv.put(tr); // gửi transaction sau khi read xong
            end
        end
    endtask
endclass
```

driver

```
class scoreboard;
    mailbox mon_to_sb = new();
    int compare_cnt;
    int acc;

    bit [7:0] ref_fifo[$]; // reference FIFO model (queue)

    function new(mailbox mon_to_sb);
        this.mon_to_sb = mon_to_sb;
        this.compare_cnt = 0;
    endfunction

    task run;
        forever begin
            transaction tr;
            mon_to_sb.get(tr);
            if (tr.tr_type == "WRITE") begin
                ref_fifo.push_back(tr.data_in);
                $display("[SB] WRITE logged: data_in = %0d", tr.data_in);
                acc++;
            end
            else if (tr.tr_type == "READ") begin
                if (ref_fifo.size() > 0) begin
                    bit [7:0] expected = ref_fifo.pop_front();
                    if (expected == tr.data_out) begin
                        $display("[SB] MATCH: expected=%0d, got=%0d ", expected, tr.data_out);
                        acc++;
                    end
                    else
                        $display("[SB] MISMATCH: expected=%0d, got=%0d ", expected, tr.data_out);
                end
                else begin
                    $display("[SB] UNDERFLOW DETECTED – no data in reference FIFO!");
                end
            end
            compare_cnt++;
        end
    endtask
endclass
```

generator

3. Design Verification

Testbench top

```
`include "interface.sv"
`include "test.sv"
module tb_top;

    // Clock & reset
    bit clk;
    bit rst_n;

    // Tham số DUT
    localparam DATASIZE = 8;
    localparam DEPTH = 16;
    localparam PTR_WIDTH = $clog2(DEPTH);

    // Clock generator
    initial clk = 0;
    always #2 clk = ~clk;

    // Interface instance
    fifo_if vif(clk, rst_n);
```

```
// DUT instance
Synchronous_FIFO_memory #(DATASIZE, DEPTH, PTR_WIDTH) dut (
    .clk(vif.clk),
    .rst_n(vif.rst_n),
    .w_en(vif.w_en),
    .r_en(vif.r_en),
    .data_in(vif.data_in),
    .data_out(vif.data_out),
    .fifo_full(vif.fifo_full),
    .fifo_empty(vif.fifo_empty),
    .fifo_overflow_flag(vif.fifo_overflow_flag), // optional
    .fifo_underflow_flag(vif.fifo_underflow_flag) // optional
);

// Test / Environment
test t1(vif);

// Reset sequence
initial begin
    clk = 0;
    rst_n = 0;
    #5 rst_n = 1;

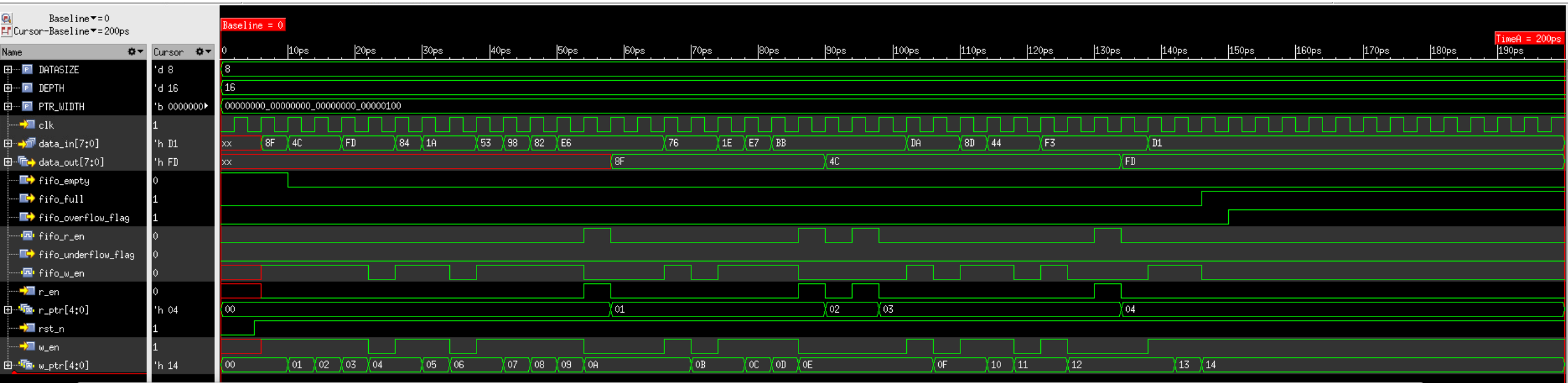
end

// Waveform dump
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(1);

end

endmodule
```

3. Design Verification



3. Design Verification



4. Synthesis

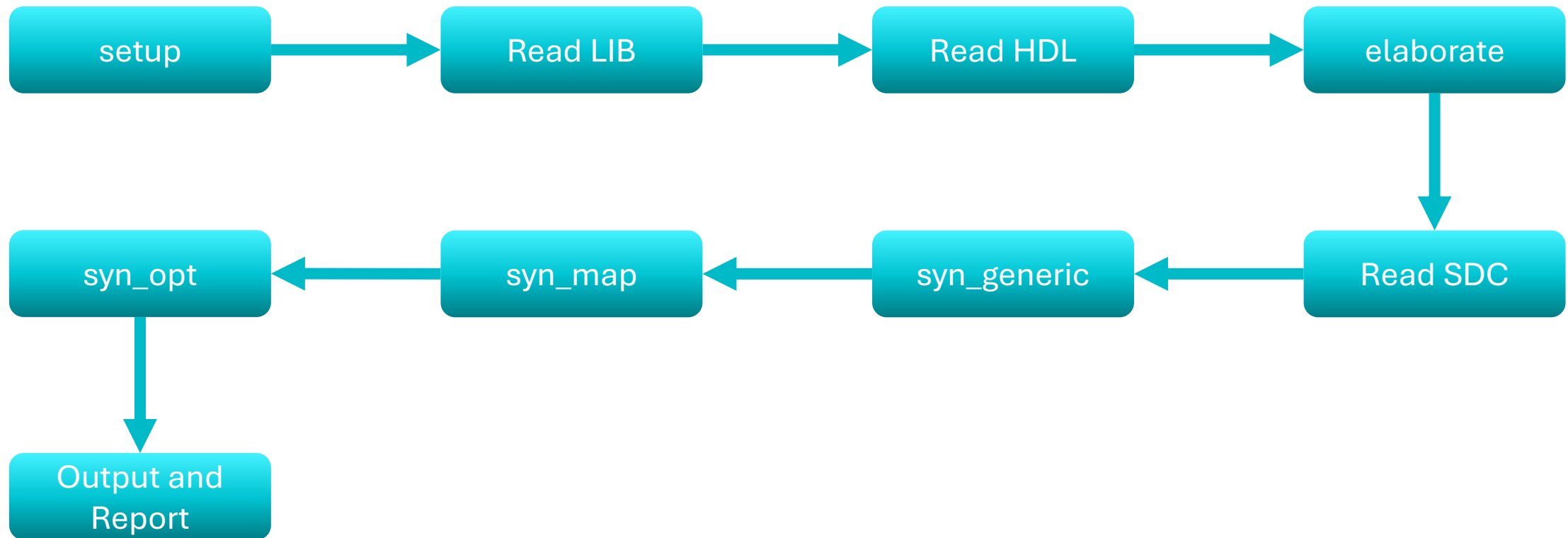
4.1. Synthesis Flow

4.2. Synthesis Design Constraints

4.3 Results

4. Synthesis

4.1. Synthesis Flow



4. Synthesis

4.1. Synthesis Flow

```
#=====
# Setup Paths
#=====
set_db init_lib_search_path ../../LIBS/lib/max
set_db init_hdl_search_path ../RTL

#=====
# Initial Settings
#=====
set_db design_process_node 45
set_db design_power_effort high
set_db hdl_preserve_unused_registers false
set_db delete_unloaded_seqs true
set_db optimize_constant_0_flops true
set_db optimize_constant_1_flops true
set_db optimize_constant_latches true

set_db syn_global_effort high
set_db retime_effort_level high
set_db retime_optimize_reset true

set_db syn_generic_effort high
set_db syn_map_effort high
set_db syn_opt_effort high
set_db opt_area_recovery true
set_db opt_fix_fanout_load true
set_db opt_area_recovery_setup_target_slack 0.05
set_db opt_all_end_points true
set_db opt_add_always_on_feed_through_buffers true
```

4. Synthesis

4.1. Synthesis Flow

```
#=====
# Read Libraries
#=====
read_libs ../../LIBS/lib/max/fast_vdd1v2_basicCells.lib \
          ../../LIBS/lib/max/pdkIO.lib \
          ../../LIBS/lib/max/pllclk_slow.lib

#=====
# Read RTL & Elaborate
#=====
read_hdl -language v2001 ../RTL/fifo.v

elaborate
write_db -all_root_attributes ../OUTPUT/fifo.elaborate.db
```

```
#=====
# Read SDC
#=====
read_sdc ../SDC/fifo.sdc
#=====
# Check design
#=====
report_timing -lint -verbose > ../REPORT/timing_lint.rpt
check_design -all > ../REPORT/check_design.rpt
```

4. Synthesis

4.1. Synthesis Flow

```
#####  
# Synthesis & Mapping  
#####  
syn_generic  
syn_map  
  
write_hdl -lec > ../OUTPUT/fifo.map.v  
write_sdc > ../OUTPUT/fifo.map.sdc  
write_db -all_root_attributes ../OUTPUT/fifo.map.db  
write_do_lec -revised_design ../OUTPUT/fifo.map.v > ./wlec_fifo_rtl2g1
```

```
#####  
# Post Optimization  
#####  
for {set i 0} {$i < 3} {incr i} {  
    puts "Optimization round: $i"  
    syn_opt  
}
```

4. Synthesis

4.1. Synthesis Flow

```
foreach cost_group [get_db cost_groups] {  
    set base_name [basename $cost_group]  
    regsub -all {\/*} $base_name "-" base_name  
    report_timing -cost_group [list $cost_group] -max_paths 100 > ../REPORT/${base_name}.final.rpt  
}  
  
report_timing -max_paths 100 > ../REPORT/report_timing.rpt  
report_power > ../REPORT/report_power.rpt  
report_area -detail > ../REPORT/report_area.rpt  
report_qor > ../REPORT/report_qor.rpt  
  
write_db -all_root_attributes ../OUTPUT/fifo.final.db  
write_hdl -lec > ../OUTPUT/fifo_netlist.v  
write_sdc > ../OUTPUT/fifo_sdc.sdc  
write_do_lec -golden_design ../OUTPUT/fifo_netlist.v -revised_design ../OUTPUT/fifo.map.v > ./wlec_fifo_gltog2  
write_sdf -timescale ns -nonegchecks -recrsm split -edges check_edge -setuphold split > ../OUTPUT/fifo_delay.sdf
```


4. Synthesis

4.2. Synthesis Design Constraints

```
# ===== CLOCK CONSTRAINTS =====

# Create main clock: 300 MHz => period = 3.34ns, 50% duty cycle
create_clock -name clk -period 3.34 -waveform {0 1.67} [get_ports clk]

# ===== I/O DELAYS =====

# Set input delay (assume 60% of duty cycle ? 2.0ns)
set_input_delay 2.0 -clock clk [get_ports data_in*]
set_input_delay 2.0 -clock clk [get_ports w_en]
set_input_delay 2.0 -clock clk [get_ports r_en]
set_input_delay 2.0 -clock clk [get_ports rst_n]

# Set output delay (assume 60% of duty cycle ? 2.0ns)
set_output_delay 2.0 -clock clk [get_ports data_out*]
set_output_delay 2.0 -clock clk [get_ports fifo_full]
set_output_delay 2.0 -clock clk [get_ports fifo_empty]
set_output_delay 2.0 -clock clk [get_ports fifo_overflow_flag]
set_output_delay 2.0 -clock clk [get_ports fifo_underflow_flag]

# ===== ASYNC RESET =====

# Treat rst_n as constant 0 for static timing
set_case_analysis 0 [get_ports rst_n]

# N?u b?n mu?n dùng false path thay cho case_analysis:
# set_false_path -from [get_ports rst_n]

# ===== DESIGN RULES (DRV) =====

# Set max transition (clock is slower so transition time có th? r?ng h?n)
set_max_transition 0.3 [get_clocks clk]
set_max_transition 0.4 [get_ports data_in*]
set_max_transition 0.4 [get_ports w_en]
set_max_transition 0.4 [get_ports r_en]

# Set max capacitance and fanout
set_max_capacitance 8 [current_design]
set_max_fanout 16 [current_design]

# Prevent usage of large power-hungry cells
set_dont_use [get_lib_cells *X12*]
set_dont_use [get_lib_cells *X16*]
set_dont_use [get_lib_cells *X20*]

# ===== CLOCK UNCERTAINTY =====

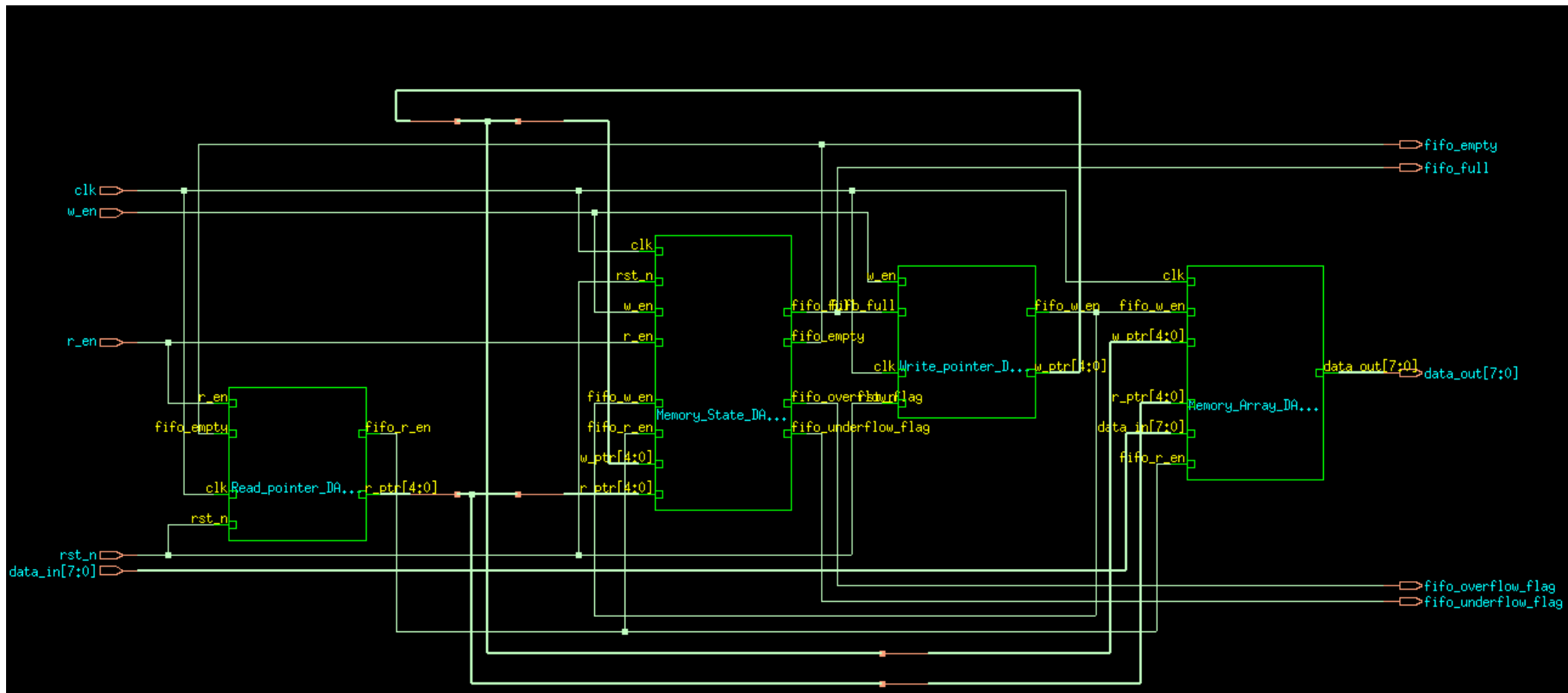
# Set clock uncertainty (can be relaxed a bit due to lower freq)
set_clock_uncertainty 0.2 [get_clocks clk]

# ===== OPTIONAL FALSE PATHS =====

# Uncomment below if some outputs are not timing-critical
# set_false_path -from [get_ports fifo_empty]
# set_false_path -from [get_ports fifo_full]
# set_false_path -from [get_ports fifo_overflow_flag]
# set_false_path -from [get_ports fifo_underflow_flag]
```

4. Synthesis

4.4. Results



4. Synthesis

4.4. Results

QoR Reports

```
=====
Generated by:      Genus(TM) Synthesis Solution 22.12-s082_1
Generated on:      Nov 01 2025  11:31:25 pm
Module:           fifo
Operating conditions: PVT_1P32V_0C (balanced_tree)
Wireload mode:    segmented
Area mode:        timing library
=====
```

Timing

Clock Period

clk 3340.0

Cost Group	Critical Path Slack	TNS	Violating Paths
clk	No paths	0.0	
default	No paths	0.0	
in2out	No paths	0.0	
in2reg	957.3	0.0	0
reg2out	981.0	0.0	0
reg2reg	2852.6	0.0	0
Total		0.0	0

Instance Count

Leaf Instance Count	320
Physical Instance count	0
Sequential Instance Count	140
Combinational Instance Count	180
Hierarchical Instance Count	0

Area

Cell Area	1332.774
Physical Cell Area	0.000
Total Cell Area (Cell+Physical)	1332.774
Net Area	0.000
Total Area (Cell+Physical+Net)	1332.774

4. Synthesis

4.4. Results

QoR Reports

```
Power
-----
Leakage Power          134.728 nW
Dynamic Power          201865.409 nW
Total Power            202000.137 nW

Max Fanout              140 (clk)
Min Fanout              0 (rst_n)
Average Fanout          3.2
Terms to net ratio      4.1805
Terms to instance ratio 4.4156
Runtime                 43.999994 seconds
Elapsed Runtime         73 seconds
Genus peak memory usage 1492.80
Innovus peak memory usage no_value
Hostname                training
```

4. Synthesis

4.4. Results

Timing Reports

Path 1 MET (957 ps) Setup Check with Pin top3_data_out2_reg_7_7/CK->SE

Group: in2reg

Startpoint: (R) w_en

Clock: (R) clk

Endpoint: (F) top3_data_out2_reg_7_7/SE

Clock: (R) clk

	Capture	Launch
Clock Edge:+	3340	0
Drv Adjust:+	0	0
Src Latency:+	0	0
Net Latency:+	0 (I)	0 (I)
Arrival:=	3340	0

Setup:- 36

Uncertainty:- 200

Required Time:= 3104

Launch Clock:- 0

Input Delay:- 2000

Data Path:- 146

Slack:= 957

Exceptions/Constraints:

input_delay 2000

fifo.sdc_line_10

#	Timing Point	Flags	Arc	Edge	Cell	Fanout	Load (fF)	Trans (ps)	Delay (ps)	Arrival (ps)	Instance Location
#	w_en	-	-	R	(arrival)	2	0.6	0	0	2000	(-, -)
#	g227/Y	-	B->Y	F	NAND2XL	4	1.2	24	15	2015	(-, -)
#	g226/Y	-	A->Y	R	INVXL	2	0.6	10	12	2027	(-, -)
	g224/Y	-	B->Y	F	NAND2XL	5	1.5	28	20	2047	(-, -)
	g288/Y	-	A->Y	R	INVXL	3	0.9	14	14	2062	(-, -)
	g241/Y	-	B->Y	F	NAND2XL	3	1.3	25	20	2081	(-, -)
	g240/Y	-	A->Y	R	INVXL	3	0.9	14	14	2095	(-, -)
	g235/Y	-	B->Y	F	NAND2XL	9	5.1	82	51	2146	(-, -)
#	top3_data_out2_reg_7_7/SE	-	-	F	SDFFX1	9	-	-	0	2146	(-, -)

4. Synthesis

4.4. Results

Power Report

Instance: /fifo

Power Unit: W

PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
register	1.07703e-07	1.74399e-04	2.79366e-06	1.77301e-04	87.77%
latch	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
logic	2.70254e-08	1.87617e-06	8.85825e-07	2.78902e-06	1.38%
bbox	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
clock	0.000000e+00	0.000000e+00	2.19104e-05	2.19104e-05	10.85%
pad	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
pm	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.00%
Subtotal	1.34728e-07	1.76275e-04	2.55899e-05	2.02000e-04	100.00%
Percentage	0.07%	87.27%	12.67%	100.00%	100.00%

5. Formal Verification

5. Formal Verification

5.1. Formal Verification

=====		
Summary	Golden	Revised
=====		
Design-modules	6	1
Library-cells	37	327
=====		
Primitives	Golden	Revised
=====		
INPUT	12	12
OUTPUT	12	12

AND	*	166
BUF	*	2
DFF		140
INV	*	23
MUX	*	268
NAND	*	0
NOR	*	16
OR	*	12
XOR	*	16
----- word-level -----		
ADD	*	2
SUBTRACT	*	1
WMUX	*	34
WSEL	*	1

Total	643	1027

=====				
Mapped points: SYSTEM class				

Mapped points	PI	P0	DFF	Total

Golden	12	12	140	164

Revised	12	12	140	164
=====				

Mapping method	PI	P0	DFF	Total

Name	12	12	140	164
=====				

5. Formal Verification

5.1. Formal Verification

Verification Report		
Category		Count
1. Non-standard modeling options used:		1
Tri-stated output:	checked	
Revised X signals set to E:	yes	
Floating signals tied to Z:	no *	
Command "add clock" for clock-gating:	not used	
2. Incomplete verification:		0
All primary outputs are mapped:	yes	
Not-mapped DFF/DLAT is detected:	no	
All mapped points are added as compare points:	yes	
All compared points are compared:	yes	
User added black box:	no	
Black box mapped with different module name:	no	
Empty module is not black boxed:	no	
Command "add ignore outputs" used:	no	
Always false constraints detected:	no	
3. User modification to design:		0
Change gate type:	no	
Change wire:	no	
Primary input added by user:	no	
4. Conformal Constraint Designer clock domain crossing checks recommended:		0
Multiple clocks in the design:	no	

6. Compare Results:	PASS
Number of EQ compare points:	152
Number of NON-EQ compare points:	0
Number of Aborted compare points:	0
Number of Uncompared compare points :	0
pass	

Thanks for watching

Digital VLSI Design

Actor: Hoang Bao Long